

Chapter 7. 트랜스포머(~GPT)

- 트랜스포머의 주요 기능 : 병렬로 입력 시퀀스 처리 → 빠르고 효율적
- 모델 구조: 기존의 순차 처리나 반복 연결에 의존X, 셀프 어텐션 기반
- 오토 인코딩, 자기 회귀 방식 또는 두 개의 조합으로 학습
 - 오토 인코딩: 랜덤하게 문장의 일부를 빈칸 토큰으로 만들고 해당 빈칸에 어떤 단어가 적절한지 예측하는 작업 수행
 - 예측되는 토큰의 양옆에 있는 토큰들을 참조하기 때문에 양방향 구조를 가짐 → 인코더
- 트랜스포머 구조



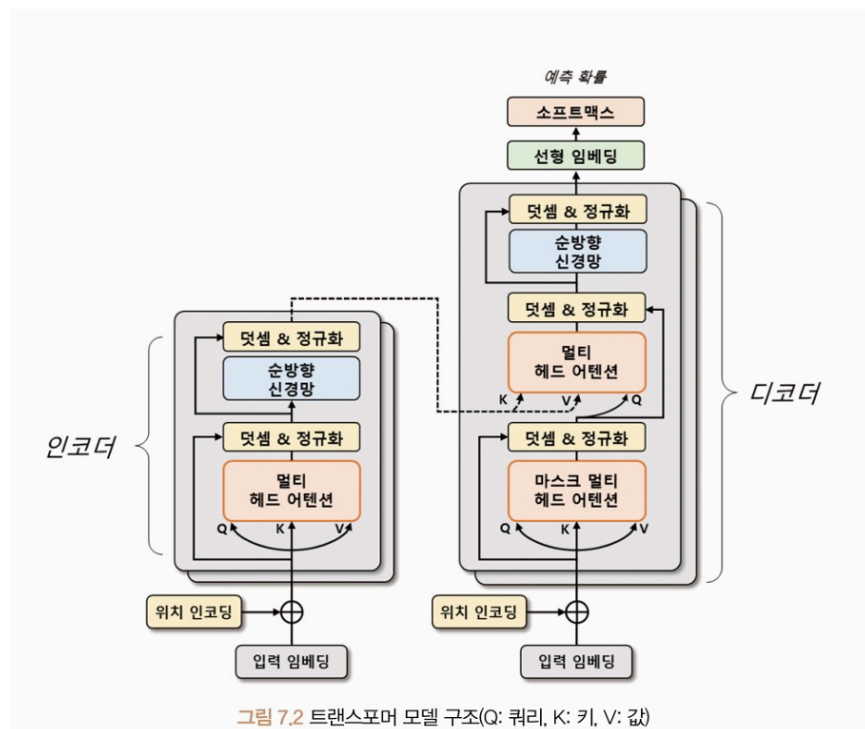
- B는 예측되는 토큰의 왼쪽에 있는 토큰들만 참조하기 때문에 단방향 구조 → 디코더
- 트랜스포머 모델 구조

모델	학습구조	학습방법	학습 방향성
BERT	인코더	오토 인코딩	양방향
GPT	디코더	자기 회귀	단방향
BART	인코더+디코더	오토 인코딩+자기 회귀	양방향+단방향
ELECTRA	인코더+판별기	오토 인코딩+대체 토큰 탐지	양방향
T5	인코더+디코더	오토 인코딩+자기 회귀+다양한 자연어 처리 작업을 학습	양방향

7.1 Transformer

딥러닝 모델 중 하나로, 다양한 자연어 처리 분야에서 성과를 내는 모델이다. 이 모델은 RNN, CNN과 달리 **어텐션 메커니즘**만을 사용하여 시퀀스 임베딩을 표현한다.

- 트랜스포머의 어텐션 메커니즘
 - 인코더와 디코더 간의 상호작용으로 입력 시퀀스의 중요한 부분에 초점을 맞추어 문맥을 이해하고 적절한 출력 생성
 - 인코더: 입력 시퀀스를 임베딩하여 고차원 벡터로 변환
 - 디코더: 인코더의 출력을 입력으로 받아 출력 시퀀스 생성
 - 어텐션 메커니즘은 인코더와 디코더 사이의 상관관계를 계산하여 중요 정보에 집중
- 장점
 - 학습 속도가 빠르며, 병렬 처리가 가능해 대규모 데이터셋에서 높은 성능
 - 임베딩 과정에서 문장 전체 정보 고려 → 문장의 길이가 길어지더라도 성능 유지
- 트랜스포머 모델 구조



- N개의 트랜스포머 블록으로 구성 → 멀티 헤드 어텐션, 순방향 신경망으로 구성
- 멀티 헤드 어텐션: 쿼리, 키, 값 벡터 정의 → 입력 시퀀스들 관계를 셀프 어텐션

- 순방향 신경망: 산출된 임베딩 벡터를 고도화하기 위해 사용
- 트랜스포머는 입력 시퀀스 데이터를 소스, 타겟 데이터로 나눠 처리
 - 인코더: 소스 시퀀스 데이터를 위치 인코딩된 입력 임베딩으로 표현 → 트랜스포머 블록의 출력 벡터 생성
 - 디코더: 마스크 멀티 헤드 어텐션을 사용해 타겟 시퀀스 데이터를 순차적으로 생성
 - 인코더의 출력 벡터 정보 참조
- 최종 생성된 디코더 출력 벡터 ⇒ 선형 임베딩으로 재표현돼, 이미지나 자연어 모델에 활용

7.1.1 입력 임베딩과 위치 인코딩

- 입력 시퀀스를 병렬 구조로 처리, 단어 순서 정보 X → 위치 정보 임베딩 벡터에 추가, 단어 순서 정보 모델에 반영
 - ⇒ 트랜스포머는 위치 인코딩 방식 사용!
- 위치 인코딩: 입력 시퀀스의 순서 정보를 모델에 전달하는 방법
 - 위치 인코딩 벡터는 sin, cos 함수 사용을 통해 생성 → 임베딩 벡터 + 위치 정보
 - 각 토큰의 위치 정보 모델이 학습

#예제 7.1 위치 인코딩

- PositionalEncoding 클래스: 입력 임베딩 차원, 최대 시퀀스를 입력 받는다
- 입력 시퀀스의 위치마다 sin과 cos 함수로 위치 인코딩을 계산한다
- 위치 인코딩 계산 방식

$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$

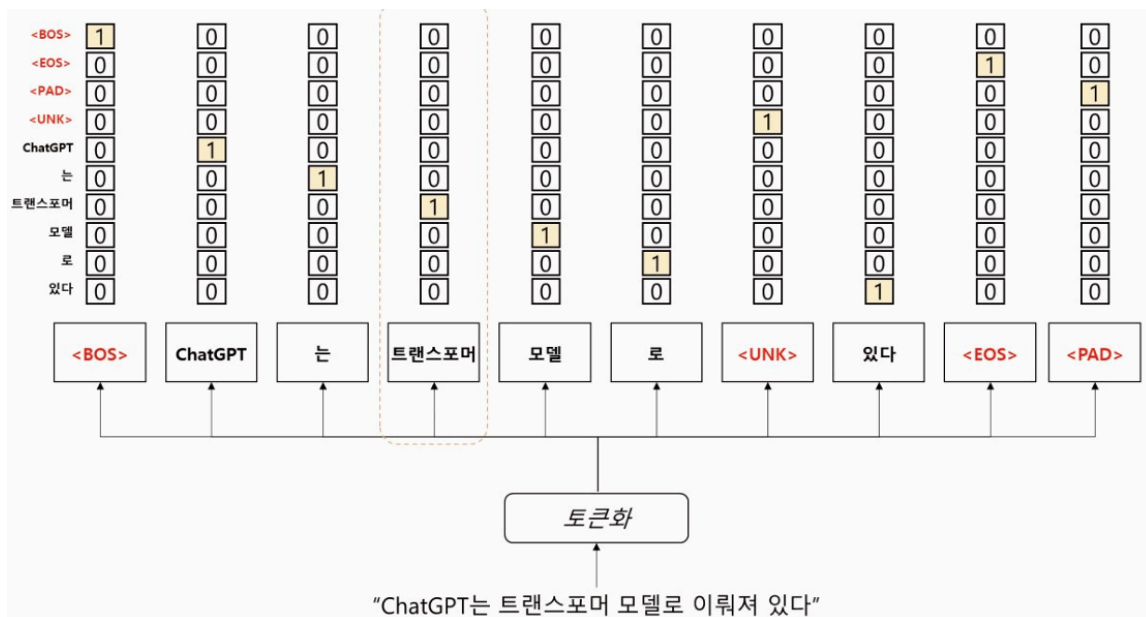
$$PE(pos, 2i + 1) = \cos(pos/10000^{2i/d_{model}})$$

- pos: 입력 시퀀스에서 해당 단어의 위치
- i: 임베딩 벡터의 차원 인덱스
 - 차원의 인덱스가 짝수라면 첫 번째 sin 함수 수식 적용

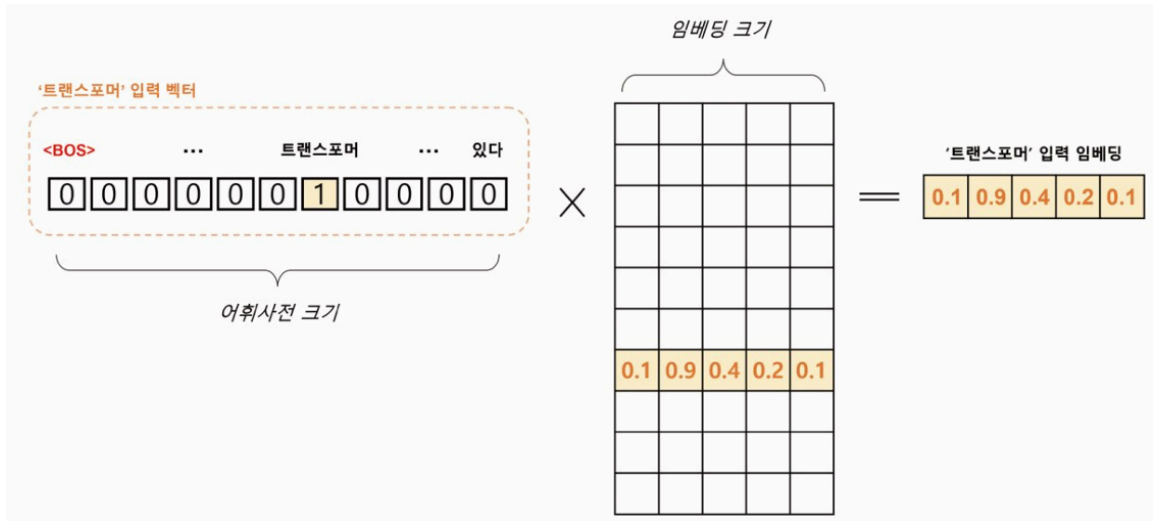
- 홀수라면 두 번째 $\cos$ 함수 수식 적용
 - $1/10000^{2i/d}$ 는 각도 정보 변환을 위한 스케일링 인자, 각 위치에 대한 주기적 신호 생성
 - pe 변수의 텐서 차원 $[50, 1, 128] \rightarrow$ [최대 시퀀스, 1, 입력 임베딩의 차원]
- register_buffer 메서드: 모델이 매개변수 갱신X 설정

7.1.2 특수 토큰

- 트랜스포머는 단어 토큰 이외의 특수 토큰을 활용하여 문장 표현
- 특수 토큰: 입력 시퀀스의 시작과 끝을 나타내는 마스킹 영역으로 사용
- 모델이 입력 시퀀스의 시작과 끝을 인식, 마스킹을 통해 일부 입력 무시
- 특수 토큰을 활용한 문장 토큰 배열



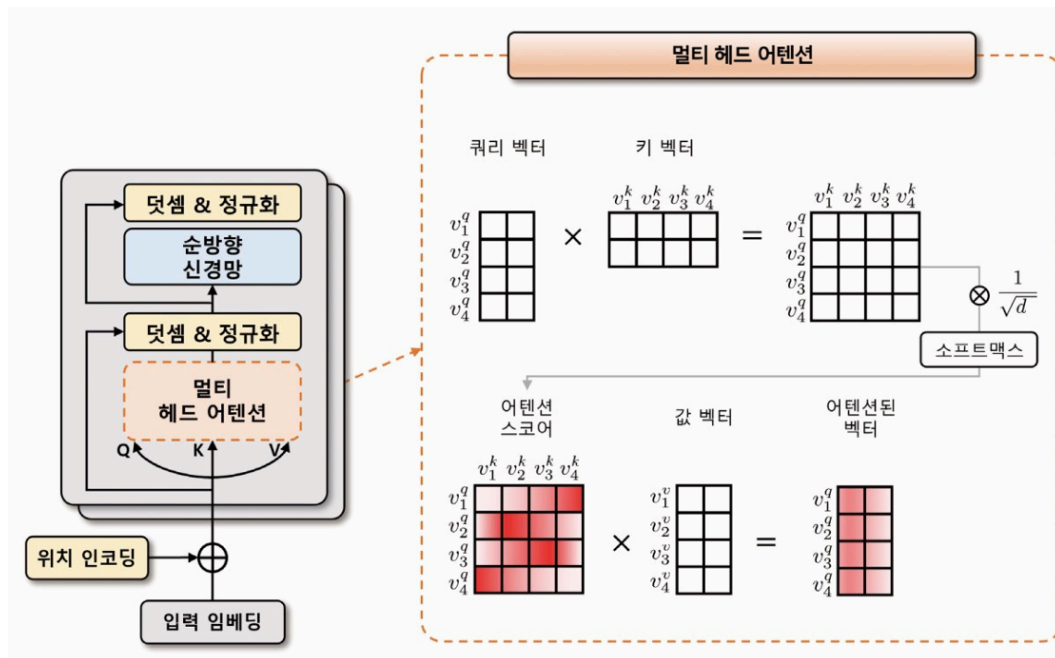
- BOS(Begining of Sentence): 문장의 시작
 - EOS(End of Sentence): 문장의 종료
 - UNK토큰: 어휘 사전에 없는 단어로, 모르는 단어
 - PAD 토큰: 모든 문장을 일정한 길이로 맞추기 위해 사용, 짧은 문장의 빈공간을 채울 때 사용
- 입력 임베딩 산출과정



- 어휘 사전 크기 V , 입력 임베딩 차원 d
- '트랜스포머'의 원-핫 벡터는 $[1, V]$ 크기
- N 개의 문장이 최대 S 개의 토큰 길이를 가질 때 $[N, S, V]$ 크기의 원-핫 벡터 텐서는 $[N, s, d]$

7.1.3 트랜스포머 인코더

- 입력 시퀀스를 받아 여러 개의 계층으로 구성된 인코딩 계층을 거쳐 연산 수행
- 각 인코더 계층은 멀티 헤드 어텐션과 순방향 신경망으로 구성
- 입력 데이터에 대한 정보를 추출, 다음 계층에 전달
- 인코더 계층에서 위치 정보를 반영하기 위해 위치 임베딩 벡터를 입력 벡터에 더해줌
- 산출된 인코더 계층의 출력은 디코더 계층으로 전달
- 트랜스포머 인코더 블록의 연산 과정



- 멀티 헤드 어텐션 입력 텐서 차원 $[N, s, d] \rightarrow$ 입력 임베딩은 선형 변환을 통해 3개의 임베딩 벡터 생성, 각각 쿼리(Q), 키(K), 값(V) 벡터로 정의
- 쿼리 벡터: 현재 시점에서 참조하고자 하는 정보의 위치를 나타내는 벡터
- 키 벡터: 쿼리 벡터와 비교되는 대상으로 쿼리 벡터를 제외한 입력 시퀀스에서 탐색되는 벡터가 됨
- 값 벡터: 쿼리 벡터와 키 벡터로 생성된 어텐션 스코어를 얼마나 반영할지 설정하는 가중치
- 쿼리, 키 벡터를 사용한 어텐션 스코어 계산식

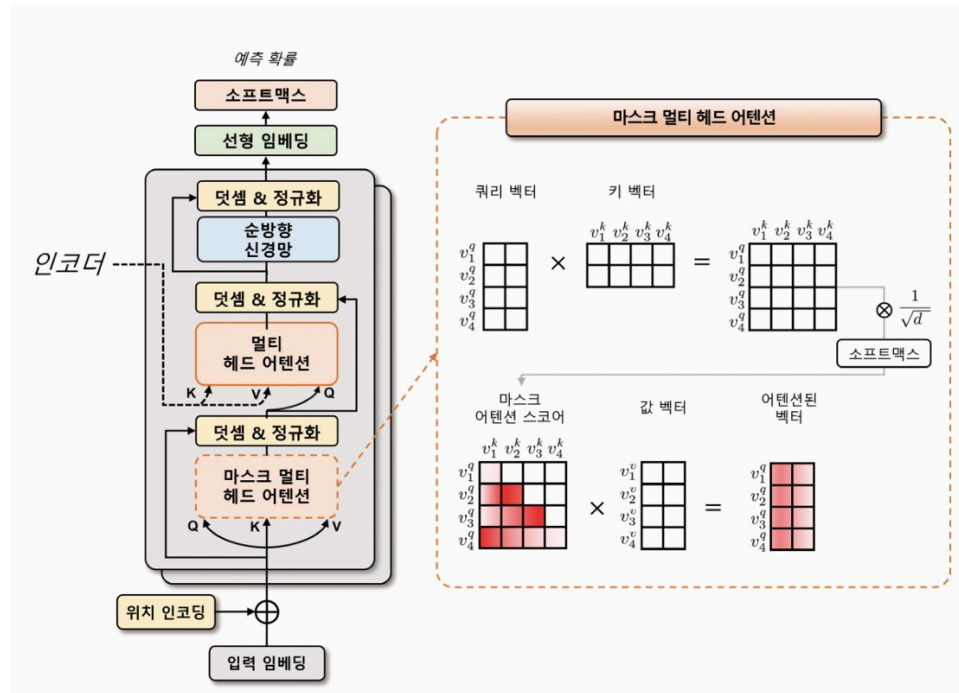
$$score(v^q, v^k) = \text{softmax}\left(\frac{(v^q)^T \cdot v^k}{\sqrt{d}}\right)$$

- 보정된 어텐션 스코어를 소프트맥스 함수를 이용해 확률적으로 재표현
- 값 벡터와 내적하여 셀프 어텐션된 벡터 생성 $\rightarrow$ 반복 $\rightarrow$ 스코어 맵 생성
- 멀티 헤드: 이러한 셀프 어텐션을 여러 번 수행해 여러 개의 헤드
 - 각각의 헤드가 독립적으로 어텐션 수행, 결과 합침
- 텐서는 k개의 셀프 어텐션된 $[N, S, d/k]$ 텐서를 의미
- k개의 셀프 어텐션 벡터는 임베딩 차원 축으로 다시 병합되어 $[N, S, d]$ 의 형태 출력
- 멀티 헤드 어텐션 통과 이전 입력값과 이후 출력값 텐서를 더함 $\rightarrow$ 기울기 소실 완화

- 임베딩 차원 축으로 정규화하는 계층 정규화 적용
- 순방향 신경망: 선형 임베딩과 ReLU로 이뤄진 인공 신경망이나 1차 합성곱 사용
- 트랜스포머 인코더는 여러 개의 트랜스포머 인코더 블록으로 구성
 - 이전 블록에서 출력된 벡터는 다음 블록의 입력으로 전달되어 인코더 블록을 통과하면서 점차 입력 시퀀스 정보 추상화
- 인코더 블록에서 출력된 벡터는 디코더에서 사용
 - 디코더의 멀티 헤드 어텐션 모듈에서 참조되는 키, 값 벡터로 활용

7.1.4 트랜스포머 디코더

- 위치 인코딩이 적용된 타깃 데이터의 입력 임베딩을 입력 받음
- 위치 인코딩은 입력 시퀀스 내에서 각 단어의 상대적 위치 정보를 전달하는 기법
 - 디코더의 입력 임베딩에 위치 정보를 추가함으로써 입력 시퀀스 순서 정보 학습 가능
- 트랜스포머 모델에서 인코더의 멀티 헤드 어텐션 모듈은 인과성을 반영한 마스크 멀티 헤드 어텐션 모듈로 대체됨
- 첫 번째 쿼리 벡터: 첫 번째 키 벡터만을 바라보게끔 마스크를 씌움
- 두 번째 쿼리 벡터: 첫 번째와 두 번째 키 벡터를 바라보게 마스크를 씌움
- 마스크 적용 → 셀프 어텐션에서 현재 위치 이전의 단어들만 참조할 수 있게 되며, 인과성 보장
- $-\infty$ 마스크를 더해줌으로써 해당 영역의 어텐션 스코어값이 0에 가까워짐
- 소프트맥스 계산 시 해당 영역의 어텐션 가중치가 0: 마스크 영역 이전 입력 토큰 정보 참조 X
- 트랜스포머 디코더 블록 연산과정



- 타깃 데이터가 쿼리 벡터로 사용, 인코더의 소스 데이터가 키와 값 벡터로 사용
 - 쿼리 벡터: 타깃 데이터의 위치 정보를 포함한 입력 임베딩과 위치 임베딩을 더한 벡터
- 어텐션 스코어 맵 계산, 소프트맥스 함수 적용 → 어텐션 가중치 계산
- 어텐션 가중치와 값 벡터를 가중합하여 멀티 헤드 어텐션의 출력 벡터를 얻음
- 디코더에서는 셀프 어텐션 방지를 위해 마스킹 적용
- 여러 개의 트랜스포머 디코더 블록으로 구성
- 이전 시점에서의 정보를 현재 시점에서 활용 가능

7.1.5 모델 실습

트랜스포머 모델을 활용해 영어-독일어 번역 모델 구성하기

- 자연어 처리를 위한 대규모 다국어 데이터 세트 중 Multi30k 데이터 셋 사용
 - 영어-독일어 병렬 말뭉치이며 토치 데이터와 토치텍스트 라이브러리로 로드 가능
 - 토치 데이터 라이브러리: 대규모 데이터셋을 다루기 쉽게 데이터를 불러오고 변환 및 배치하는 간단하고 유연한 API 제공
 - 토치 텍스트 라이브러리: 파이토치를 위한 텍스트 처리 라이브러리
 - 포르타락커: 파이썬에서 파일 락을 관리하기 위한 라이브러리

#예제 7.2 데이터셋 다운로드 및 전처리

- 각각 말뭉치에 대해 각각 토큰라이저와 어휘 사전 생성
- `get_tokenizer`: 사용자가 지정한 토큰라이저를 가져오는 유틸리티함수로 `spaCy` 라이브러리로 사전 학습된 모델을 가져온다
- `vocab_transform`: 토큰을 인덱스로 변환시키는 함수 지정
- `build_vocab_from_iterator`: 생성된 토큰을 이용해 단어 집합 생성
- `specials`: 특수 토큰 지정, `special_first` 매개변수가 참이면 특수 토큰을 단어 집합 맨 앞에 추가
- `set_default_index`: 인덱스의 기본값 설정, 어휘 사전에 없는 토큰은 `<unk>`

#예제 7.3 트랜스포머 모델 구성

- `Seq2SeqTransformer`: `TokenEmbedding` 클래스로 소스 데이터와 입력 데이터를 입력 임베딩으로 변환하여 `src_tok_emb`와 `tgt_tok_emb` 생성
- 소스와 타겟 데이터의 어휘 사전 크기를 입력받아 트랜스포머 임베딩 크기로 변환
- 트랜스포머 블록: 파이토치에서 제공하는 트랜스포머 클래스 적용
- `endoder_layers`: 인코더와 디코더
- `generator`: 마지막 트랜스포머 디코더 블록에서 산출되는 벡터를 선형 변환해 어휘 사전에 대한 로짓 생성
- 트랜스포머 클래스

```
transformer = torch.nn.Transformer(  
    d_model=512, #트랜스포머 모델의 입력과 출력 차원 크기 정의  
    nhead=8, #멀티 헤드 어텐션의 헤드 개수 정의  
    num_encoder_layers=6, #인코더 계층 수  
    num_decoder_layers=6, #디코더 계층 수  
    dim_feedforward=2048, #순방향 신경망의 은닉층 크기 정의  
    dropout=0.1, #인코더와 디코더 계층에 적용되는 드롭아웃 비율 적용  
    activation=torch.nn.functional.relu, #순방향 신경망에 적용되는 활성화 함수  
    layer_norm_eps=1e-05, #계층 정규화를 수행할 때 분모에 더해지는 입실론
```

값 정의
)

- 트랜스포머 순방향 메서드

```
output = transformer.forward(  
    src, #인코더 시퀀스  
    tgt, #디코더 시퀀스  
    src_mask=None,  
    tgt_mask=None,  
    memory_mask=None, #인코더 출력의 마스크  
    src_key_padding_mask=None,  
    tgt_key_padding_mask=None, #소스, 타겟, 메모리 시퀀스에 대한 패딩 마  
스크  
    memory_key_padding_mask=None,  
)
```

#예제 7.4 트랜스포머 모델 구조

- Seq2SeqTransformer 클래스: 입력 임베딩, 위치 임베딩, 트랜스포머블록, 로짓생성 구성
- 무시되는 색인 값을 패딩 토큰을 할당해 모델이 학습하는 동안 무시해야할 클래스 레이블 지정
- 패딩 토큰: 모델 학습에 사용X, 해당 토큰 레이블 무시, 모델이 해당 클래스 학습하지 않게 함

#예제 7.5 배치 데이터 생성

- sequential_transforms: 여러 개의 전처리 함수를 인자로 받아 이를 차례로 적용하는 함수 반환하는 함수
- 첫 번째 매개변수: token_transform 적용해 문장 토큰화
- 두 번째로 vocab_transform 적용해 각 토큰 인덱스화
- 마지막으로 적용되는 input_transform 함수는 인덱스화된 토큰에 문장의 시작과 끝을 알리는 토큰 할당

- 데이터세트를 데이터로더에 적용, 집합 함수에 `collator` 함수 적용
 - 배치 단위로 데이터 처리
- 패딩 시퀀스 함수 사용: 소스와 타겟 시퀀스 패딩

#예제 7.6 어텐션 마스크 생성

- 마스크 생성 함수: 입력으로 정수 `s`를 받아 `s*s` 크기의 마스크 생성
- `torch.triu` 함수 적용하여 상삼각행렬 생성
- 이 마스크 텐서에서 0인 값은 `-inf`로, 1인 값은 `0.0`으로 채워 어텐션 연산 적용
 - `0.0`: 셀프 어텐션에 참조되는 시퀀스
 - `-inf`: 해당 타겟 입력 시퀀스를 제외시키는 역할
- `create_mask`: 시퀀스를 입력 받아 길이 계산, 마스크 생성 함수로 타겟 시퀀스 마스크 생성
- 소스 마스크: 소스 시퀀스 길이의 크기로 채워진 행렬 생성
- 패딩 마스크: 각 시퀀스에 대해 패딩 토큰의 위치를 갖고 전치
- `source_mask`: 셀프 어텐션 과정에서 참조되는 소스 데이터의 시퀀스 범위
 - `False`: 셀프 어텐션에 참조되는 토큰
 - `True`: 어텐션에서 제외되는 토큰
- `target_mask`: [쿼리 시퀀스 길이, 키 시퀀스 길이]
- 모델이 현재 예측하고자 하는 위치 이전의 토큰들만 참고하게 제한
 - 모델이 미래 시점 정보를 사용하지 않게 현재 시점에 영향 X

#예제 7.7 모델 학습 및 평가

- `run` 함수: 모델 학습 및 평가를 위한 함수
- `create_mask` 함수: 트랜스포머 모델에 필요한 입력 패딩 마스크, 어텐션 마스크 생성

#예제 7.8 트랜스포머 모델 번역 결과

- 모델 번역 방식: 그리드 디코딩

- 디코더 네트워크가 생성한 확률 분포에서 가장 높은 확률을 가지는 단어를 선택하는 방법
- 가장 높은 확률이 높은 단어 선택 후 디코딩
- 디코더에 참조되는 마지막 인코더 트랜스포머 블록의 벡터, 타겟 데이터의 입력 텐서, 타겟 마스크를 사용하여 모델을 추론
- memory 텐서 생성: 소스 문장은 `source_tensor` 생성 후 0으로 설정
- `model.encode()`: `source_tensor`와 `source_mask`를 입력
- `max_len` 이전이거나 `EOD_IDX` 예측할 때까지 반복, 최종적으로 예측된 토큰 시퀀스 반환
- 일반적으로 생성된 문장의 길이가 소스 문장 길이 보다 약간 더 길어서 5를 더함
- `greedy_decode`: 현재까지 예측된 토큰들을 이용해 다음 토큰 예측
- 예측된 토큰 인덱스를 어휘 사전의 `lookup_tokens` 함수를 통해 텍스트로 변환
- 번역 결과
 - 이글루는 OOV 데이터로 학습하지 않았기 때문에 결과가 부정확할 가능성
 - 이글루 대신 건물을 입력해 번역하면 입력 문장의 의미를 올바르게 파악해 자연스러운 번역 결과 도출
 - 더 정확한 번역을 위해 더 많은 학습 데이터 사용 or 모델의 구조/하이퍼파라미터 등 변경

7.2 GPT

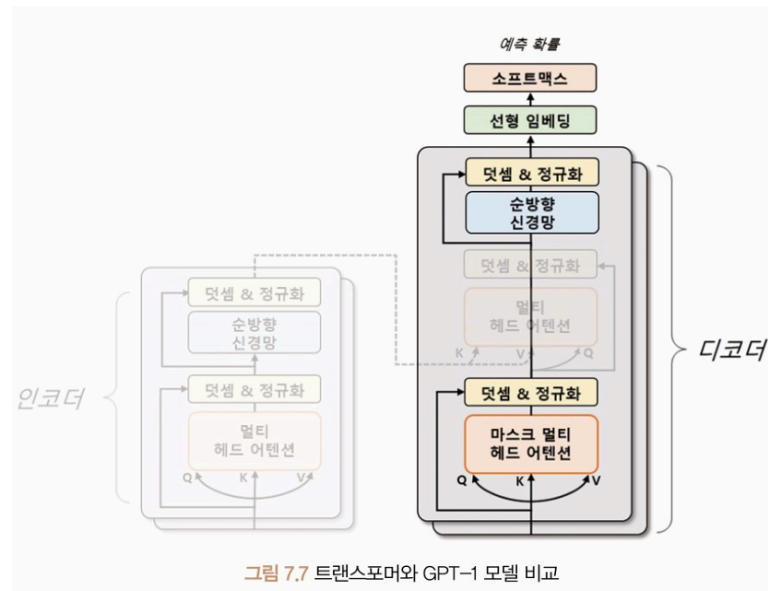
- 트랜스포머 기반 언어 모델로 가장 널리 알려진 언어 모델
- 대규모 말뭉치로 사전 학습된 모델로 다양한 다운 스트림 작업에서 우수한 성능
- 트랜스포머의 디코더를 여러 층 쌓아 만든 언어 모델
- 대규모 텍스트 데이터셋으로 사전 학습해 초기화되며, 특정 자연어 처리 작업에 맞게 미세 조정
- GPT 모델은 다양한 시리즈가 존재하며, 이 모델들들은 거의 동일한 트랜스포머 구조를 사용하며 버전이 갱신될수록 더 많은 트랜스포머 계층과 데이터를 활용해 학습 했다
- 많은 양의 매개변수를 줄이기 위해 OpenAI는 RLHF방법을 도입해 3.5 모델 학습

- 인간 피드백을 이용하는 강화 학습 방법

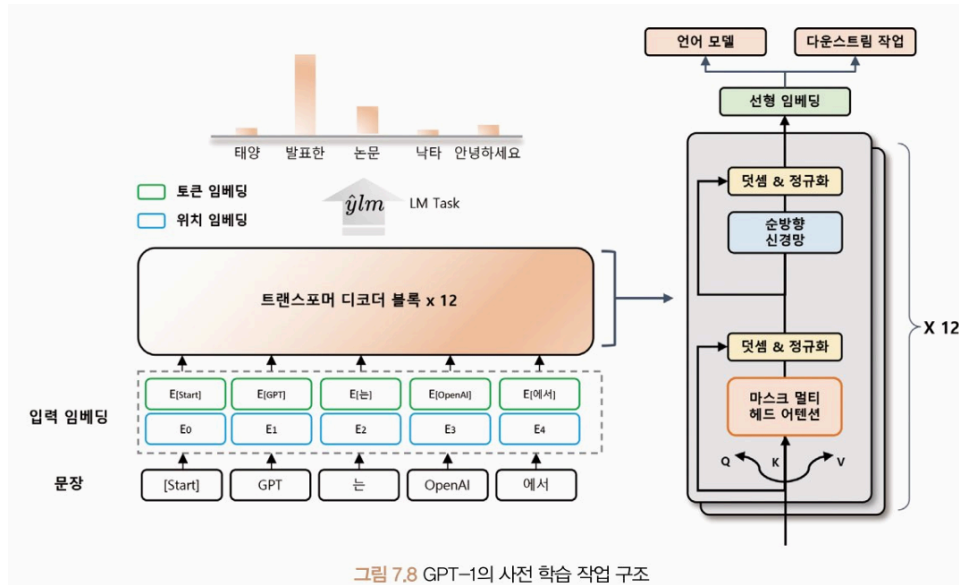
7.2.1 GPT-1

트랜스포머 구조를 바탕으로 한 단방향 언어 모델

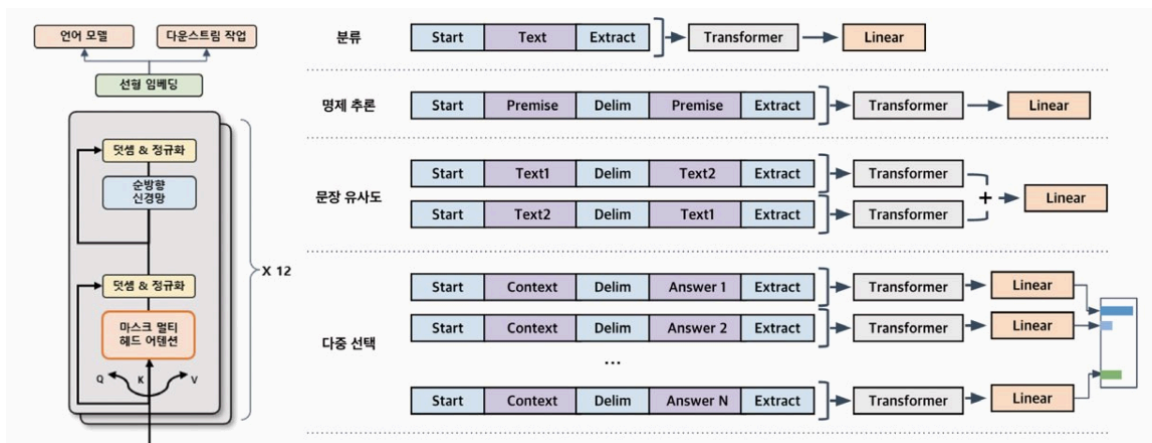
- 텍스트를 생성할 때 현재 위치에서 이전 단어들에 대한 정보만 참고해 다음 단어 예측
- 장점: 학습 속도가 빠르고, 모델 크기가 작아질 수 있음
- 트랜스포머 구조 중 디코더 부분만 사용, 인코더-디코더 간 어텐션 연산 X
- GPT-1의 트랜스포머 모델에서 GPT에 활용되는 모듈



- 디코더의 두 번째 다중헤드 어텐션 사용 X
- 12개의 헤드를 가진 트랜스포머 디코더를 12층 쌓아 모델을 구성
- 약 4.5GB의 BookCorpus 데이터셋을 사용해 언어 모델 사전 학습
 - 입력 문장을 임의의 위치까지 보여주고, 뒤에 이어지는 문장을 모델이 자동 예측
 - 이 과정에서 별도의 레이블 필요 X → 비지도 학습으로 학습이 이뤄짐
- GPT-1의 사전 학습 작업 구조



- 입력 문장을 일정한 길이의 블록으로 나누어 처리하고, 각 블록에서는 블록 내의 첫 번째 토큰부터 순서대로 다음 토큰 예측
→ 모델이 장기적인 문맥 이해 가능
- 입력 임베딩: 토큰 임베딩과 위치 임베딩을 이용하여 계산
 - 토큰 임베딩: 각 토큰을 고정된 차원의 벡터로 매핑
 - 위치 임베딩: 입력 문장에서 각 토큰의 위치 정보를 나타내는 벡터를 매핑
- 미세 조정을 위한 다운스트림 작업에서도 각 작업에 따라 입출력 구조 변화X, 언어 모델을 보조학습해 사용 가능
 - 보조학습: 모델이 주어진 주요 학습 작업 외에 다른 작은 부가적인 학습 작업 동시 수행
- GPT-1 모델 구조, 다운스트림 작업별 입출력 구조

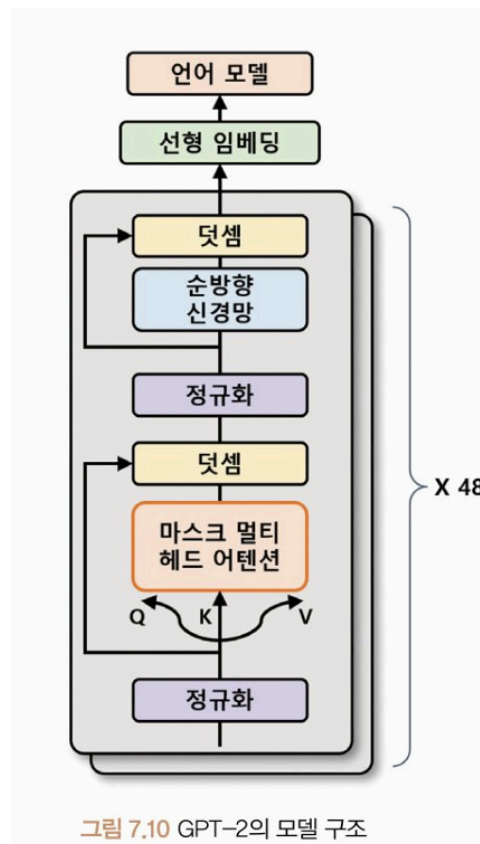


- 다운스트림 작업 → 세밀한 조정 필요: 손실 함수 외 추가 손실함수 필요
- 다운스트림 작업에서의 특수 토큰 사용: <start>, <delim>, <extract>

7.2.2 GPT-2

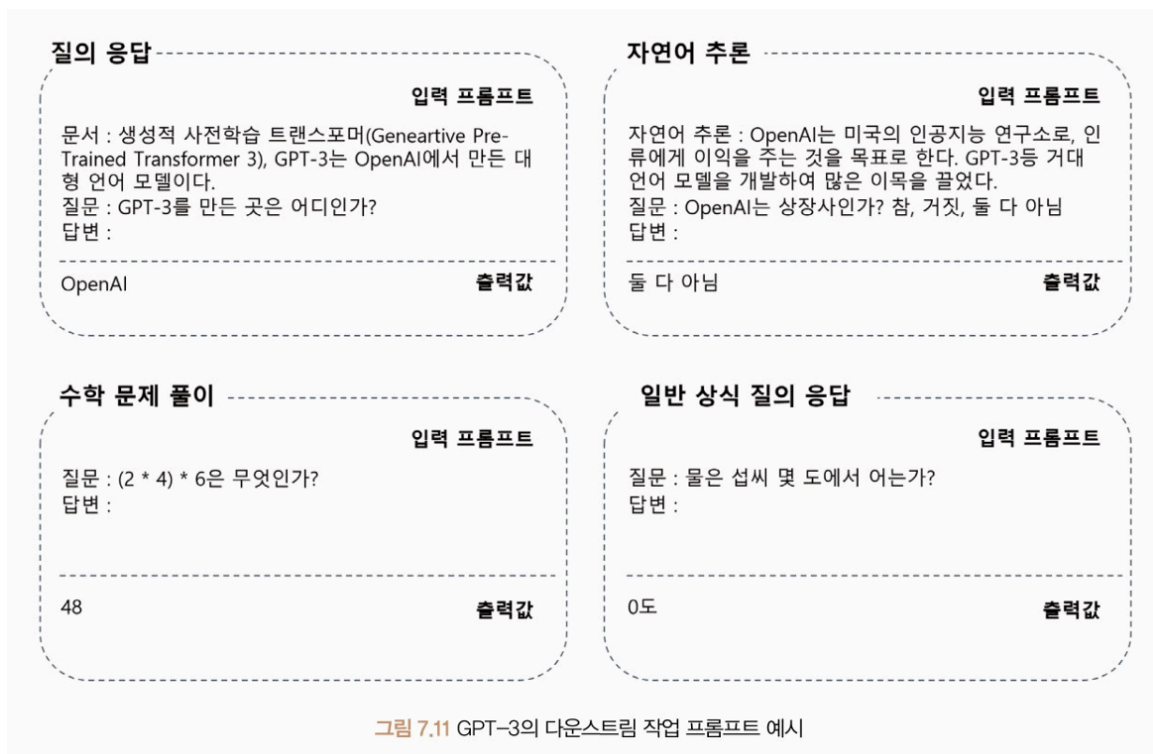
GPT-1 보다 개선된 점

- 입력 문장: 최대 512 → 1,024길이
- 디코더 층: 12개 → 48개 ⇒ 복잡한 패턴 학습 가능
- 매개변수: 1,200만 개 → 15억 개
- 제로-샷 학습: 별도의 미세 조정X, 다운스트림 작업에서도 사용 가능, 사전 학습 과정에서 특정한 형식의 데이터 입력
- GPT-2모델 구조



7.2.3 GPT-3

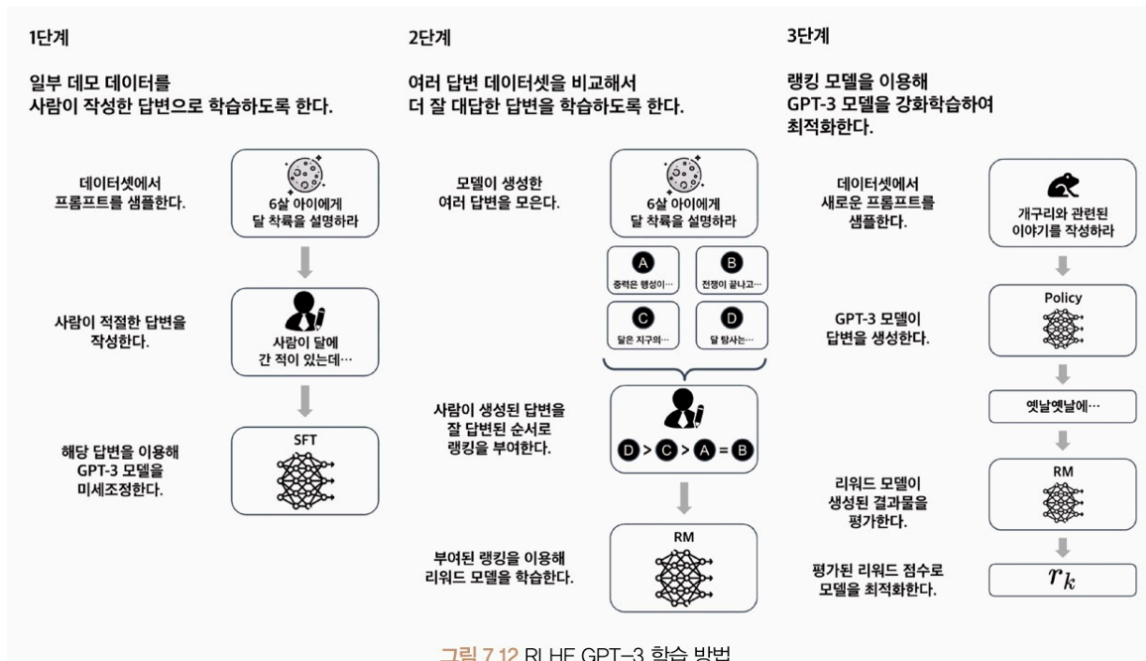
- GPT-2와 거의 동일한 모델 구조 그러나 약 116배 많은 매개변수
- 96개의 멀티 헤드 어텐션 사용, 96개의 디코더 층
- 희소 어텐션 + 일반 어텐션 : 멀티 헤드 어텐션 과정에서 연산량 감소
- 토큰 임베딩 크기: 1,600 → 12,888
- GPT-2와 같이 프롬프트 형식으로 작업 수행



- 장점
 - 높은 일반화 능력
 - 질의응답, 수학 문제 풀이, 자연어 추론 등 이전의 기술적 한계 극복
- 단점
 - 큰 모델이어서 매우 느리고 비용이 많이 듦
 - 사전 학습된 데이터 기반 작동이므로 데이터 편향 및 부정확 문제
 - 인간적인 이해력과 상식적 추론 능력 부족
 - 모델이 생성한 텍스트는 항상 사실X, 편견이나 혐오 표현 존재 가능성

7.2.4 GPT-3.5

- GPT-3의 모델 구조를 그대로 따르면서, 새로운 학습방법(RLHF) 도입



- 지도 미세 조정(SFT): 보상을 바탕으로 강화 학습을 통해 모델 학습
- RLHF에서는 주어진 프롬프트에 대해 GPT-3 언어 모델이 답변을 생성하고, 이를 이용해 사람의 피드백을 기반으로 한 학습된 보상 모델을 평가한다
- 높은 성능 향상 but 일부 입력에 대해 현실적이지 않은 결과를 생성하는 환각 현상 존재
 - 환각 현상: 모델이 인간과 같은 추론과 판단 능력을 갖추기 못하고 부적절하거나 오류가 많은 답변을 제시하는 현상

7.2.5 GPT-4

- 이전 GPT 모델과 달리, 텍스트 데이터 뿐만 아니라 이미지 데이터도 인식 가능한 **멀티 모달** 모델
- 입력 토큰: 최대 4,096(GPT-3.5) → 32,768
 - 3000개 단어 → 25,000개 단어
- 대규모 다중 작업 언어 이해 벤치마크 테스트 결과, 높은 성능 향상
- ChatGPT Plus나 마이크로소프트 Bing을 통해 모델 사용 가능

7.2.6 모델 실습

#예제 7.9 문장 생성을 위한 GPT-2 모델의 구조

- GPT2LMHeadModel 클래스의 from_pretrained: 사전 학습된 GPT-2 모델 불러오기
- gpt2-xl: 48개 디코더 계층을 사용하는 모델
- 모델 구조: 단어 토큰 임베딩, 단어 위치 임베딩, 드롭아웃, 트랜스포머 디코더 계층, 선형 임베딩 및 언어 모델로 구성
- 트랜스포머 라이브러리: 파이프라인 함수 제공
 - 문장 분류, 문장 생성, 토큰 분류 등 다양한 작업에 대한 전처리, 모델 아키텍처, 후처리 처리

#예제 7.10 GPT-2를 이용한 문장 생성

- 파이프라인 클래스: 입력된 작업에 모델로 적합한 파이프라인 구축
- 파이프라인 함수: 설정한 작업을 수행하는 파이프라인 클래스의 인스턴스 반환
 - "text-generation"(텍스트 생성 작업) 입력 → TextGenerationPipeLine 인스턴스 생성
- 입력 텍스트: 생성하려는 문장의 입력 문맥 전달
- 최대 길이: 생성될 문장의 최대 토큰 수 제한
- 반환 시퀀스 개수: 생성할 텍스트 시퀀스 수
- 패딩 토큰 ID: 모델의 자유 생성 여부 설정
 - 해당 매개변수 사용 시 모델이 입력된 문장의 문맥을 기반으로 자유롭게 다음 단어 나 문장 생성

#예제 7.11 CoLA 데이터셋 불러오기

- CoLA 데이터셋: train, dev, test로 구성
 - train: 데이터셋 모델 학습
 - dev: 데이터셋 모델 검증
 - test: 학습된 모델 성능 평가
- GPT-2는 사전 학습 시 패딩 기법 사용 X → 패딩 토큰 존재 X ⇒ eos 토큰으로 대체
- collator 함수: 배치를 토큰나이저로 토큰화하고, 패딩, 절사, 반환 형식 설정 작업 수행

- 토큰라이저: 토큰ID, 어텐션 마스크 반환
 - GPT 모델은 어텐션 마스크를 이용해 입력 문장에서 패딩 부분을 무시하고 실제 단어 처리
 - 토큰ID: 각 토큰에 대해 부여한 숫자 ID
 - 어텐션 마스크: 입력 문장의 실제 단어에 대응하는 부분으로 1, 패딩은 0

#예제 7.12 GPT-2 모델 선정

- 기본 GPT-2모델과 유사하게 작동, 최종 출력 계층이 분류를 위해 미세 조정됨
- CoLA: 올바른 문장과 올바르지 않은 문장이 태깅 → 분류 레이블 수 2로 설정

#예제 7.13 GPT-2 모델 학습 및 검증

- GPT-2 문장 분류 모델 클래스로 학습: 내부적으로 손실을 계산해 반환 → train 함수로 모델 학습 시 손실은 모델 출력값의 loss 속성으로 가져옴
→ train 함수로 모델 학습 시 손실은 모델 출력값의 loss속성으로 가져옴
- 교차 엔트로피 함수로 모델 평가: 로짓값과 레이블로 손실 계산
- 모델 선택 방법: 초기 best_loss를 큰 값으로 설정한 후, 각 에폭마다 검증 손실값이 그 값보다 작으면 모델이 개선됐다고 판단해 해당 시점의 모델 가중치 저장

#예제 7.14 모델 평가

- 손실 및 정확도가 중간 정도의 수치 : 대체 예측하지만 개선 필요
- 이 모델은 작은 데이터세트에서도 일반화 능력을 갖고 있음을 시사